SECTION 4

Setup & Reproduction

The authors' code, every case it runs, and our reimplementation of its reduced experiment



Part 1 · Authors' structure
how the files are organised and how they follow the paper

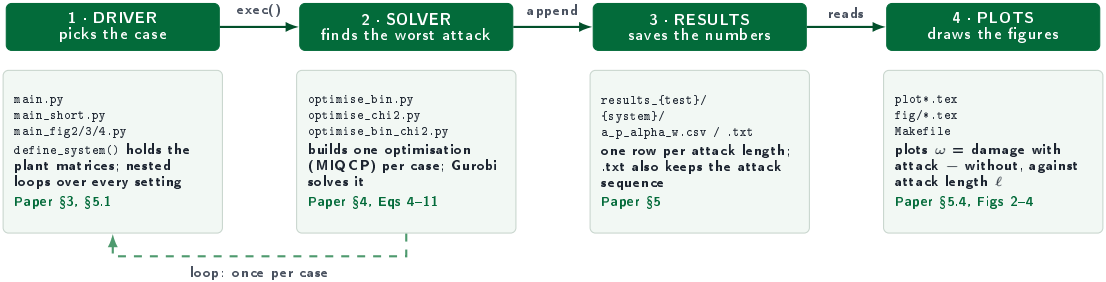
Part 2 · All related cases
every plant, detector and setting, and the one we zoom in on

Part 3 · The code
`main_short.py`: flowchart, worked example, reproduced Fig. 2

Source: Nauta, Sandberg & Maggio, ICCPS 2025 · REP code: doi.org/10.5281/zenodo.14792914 · DOI 10.1145/3716550.3722013

How the authors organised the code

Four layers, run left to right: a driver picks a case, a solver finds the worst attack, the results are saved, and plots draw the paper's figures.



One driver per figure
same code, only the loop values change

One solver per detector
`bin_chi2` = both tests together

Linked by exec()
loop settings reach the solver as globals

File name = the case
`a3_p0.25_alpha0.01_w20` → $\bar{a}$, $\hat{p}_{\mathrm{m}}$, α, n_w

Every case the code can run, and our focus

Part 2 · All related cases

A case = a plant setup × a detector: 3 × 3 = 9 cases. Each cell shows which paper figure it produces.

Plant setup ↓ · Detector →	Binomial only	χ^2 only	Binomial + χ^2
`furuta_zero`: pendulum (unstable), zero	Fig 3, Fig 4	Fig 3	Fig 2, Fig 3 · our focus
`furuta_hold`: pendulum (unstable), hold	Fig 4, Table 1	main.py only	main.py only
`qt_nmp`: quad tank (stable), hold	Fig 3	Fig 3	Fig 2, Fig 3

Which driver makes which output, and what it fixes

`main.py`	full sweep	all 9 cases
`main_fig2.py`	$\bar{a}$ 1–4, n_W 20	Fig 2 (watchdog)
`main_fig3.py`	$\bar{a}$ 4, n_W 20	Fig 3 (detectors)
`main_fig4.py`	$\bar{a}$ 4, n_W 10–18	Fig 4 + Table 1
`main_short.py`	$\bar{a}$ 3, $\hat{p}_m$ 0.25, n_W 20	one curve of Fig 2

All figures use $\hat{p}_m = 0.09, 0.17, 0.25$ and $\alpha = 0.01$.

Settings swept by main.py

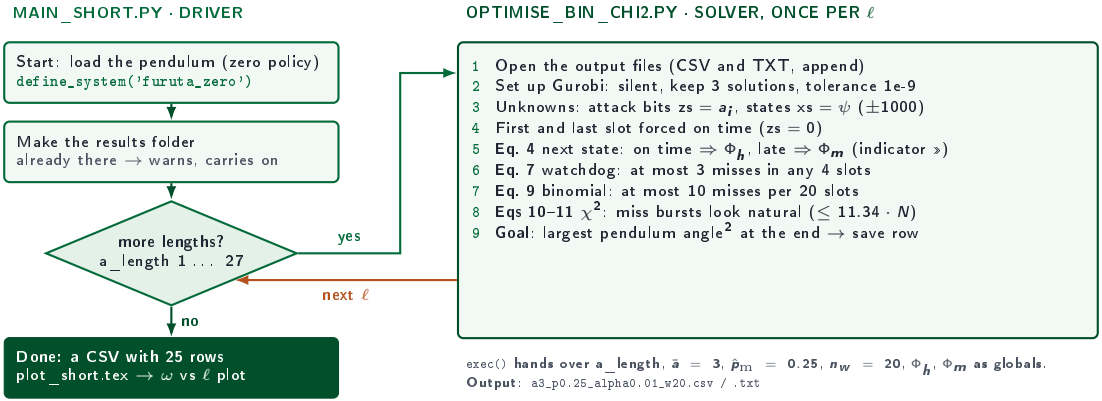
ℓ attack length 1–25 · $\bar{a}$ watchdog limit 1–5
$\hat{p}_m$ normal miss rate 0.01–0.25
n_W window 10–20 · $\alpha = 0.01$

Special cases in the code

Window $\geq$ attack → one binomial check
χ^2-only ignores window → run once at 20
Ends forced on time: ℓ = a_length − 2
Tank output = $\frac{1}{2}$ level → plots ÷ 4

OUR FOCUS · main_short.py = the highlighted cell: 27 solves vs 68,445 in main.py (∼2 h vs weeks), same solver code. *Why:* pendulum is unstable and real hardware · $\bar{a} = 3$, past which the watchdog stops mattering · $\hat{p}_m = 0.25$ gives the clearest effect · bin + χ^2 uses every constraint.

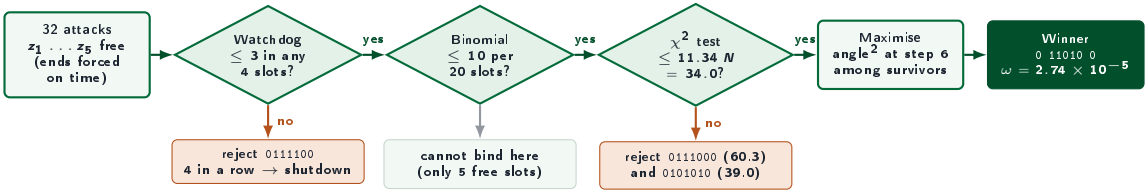
The driver tries 27 attack lengths; for each one, the solver finds the worst hidden attack and saves one row.



Worked example: how one solver call picks the attack

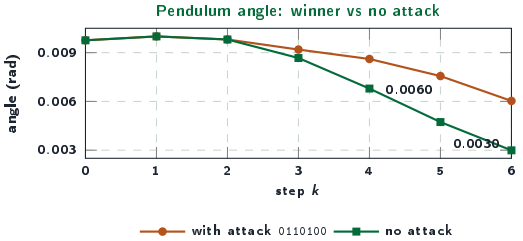
Part 3 · The code

a_length = 7 ($\ell = 5$): 32 possible attacks. Three checks remove the risky ones; the most damaging survivor wins.



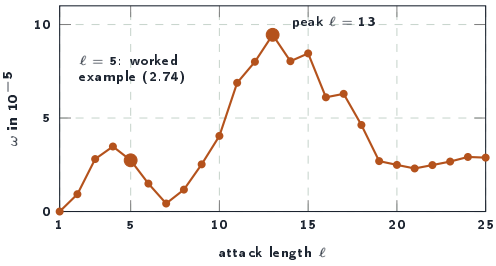
Attack	Check	angle^2	Result
0000000	no attack	0.89	baseline
0111100	4 in a row	5.75	watchdog stop
0111000	χ^2 60.3 $>$ 34.0	4.99	burst too rare
0101010	χ^2 39.0 $>$ 34.0	2.22	too regular
0110100	χ^2 19.0 $\leq$ 34.0	3.63	chosen: hidden

angle^2 at step 6, in units of 10^{-5}

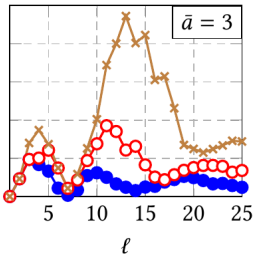


All 25 solver calls together: our reproduced curve has the same shape and peak as the paper's Fig. 2.

Ours: main_short.py output



Paper, Fig. 2 ($\bar{a} = 3$)



brown $\times$ = $\hat{p}_{\mathrm{m}}$ 0.25

What the curve shows

Short attacks exploit the initial tilt (bump at $\ell \approx 4$)
Peak at $\ell = 13$: misses interleaved with hits
Long attacks wait, then strike late

Matches the paper

same bump, dip and peak; $\ell = 5$ and $\ell = 15$ sequences identical to §5.4.1

How we reproduced it without Gurobi

Authors' matrices `define_system()` → Same constraints Eqs 7, 9, 10–11 → Try every valid attack $\ell = 1 \ldots 25$, ∼6 min → Same curve as Fig. 2 guaranteed optimum